



UNIVERSITÀ DEGLI STUDI DI SALERNO

Progettazione - GuessTheWord

Programmazione Java Avanzata

Gruppo 7

0612708848 - Emiddio Ferrentino

0612709529 - Rosa Genovese

0612708883 - Lucia Canzolino

0612708872 - Francesco Pio De Piano

A.A. 2025/26

Architettura di Sistema

L'architettura del sistema **GuessTheWord** è basata su un modello **Client-Server** distribuito, progettato per garantire un'elevata coesione interna e un basso accoppiamento tra i vari moduli. Il sistema comunica in tempo reale su rete TCP/IP.

Per essere quanto più conformi alle specifiche definite nella sezione *Analisi dei requisiti* e *Casi d'uso* è stato scelto di adottare rigorosamente il pattern **MVC (Model-View-Controller)**:

- **View:** realizzata totalmente in JavaFX utilizzando file **.fxml** progettati e strutturati con l'ausilio di SceneBuilder.
 - Questo approccio consente di disaccoppiare nettamente la logica di presentazione e il codice sorgente
- **Controller:** classi Java dedicate alla gestione degli eventi grafici e all'interazione con il livello di servizio o di rete.
- **Model:** classi di dominio che rappresentano lo stato dell'applicazione,

Per favorire la scalabilità, la manutibilità e la sicurezza del sistema, l'architettura è stata divisa in tre macro-moduli fisici e logici:

- **Modulo Server:** funge da nodo autoritativo dell'architettura. Gestisce un numero variabili di connessioni in ingresso da parte di utenti con l'ausilio del multithreading e orchestra il ciclo di vita della partita oltre all'autenticazione degli utenti sui client locali; inoltre si occupa dell'analisi dei documenti per le partite e della persistenze delle informazioni essenziali al loro svolgimento:
 - **Giocatori** coinvolti nella partita (max 2)
 - **Esito** di ciascuna partita
 - **Statistiche** pubbliche consultabili sia dall'admin e sia dai giocatori
 - **Documenti** utilizzati nell'analisi della frequenza e della rilevazione delle possibili frasi candidate in una o più eventuali partite
- **Modulo Client:** applicazione leggera che si occupa prevalentemente di fornire un'interfaccia reattiva all'utente. Fondamentalmente raccoglie gli input e li inoltra al server e non esegue alcuna logica di validazione, relegata invece al modulo Server.
- **Modulo Common:** pacchetto condiviso tra Client e Server contenenti le classi di dominio (*Utente*, *Giocatore*, *Partita*) e le infrastrutture di rete (*Messaggio*, *TipologiaMessaggio*, *Sfida*). La condivisione di questo modulo è essenziale per garantire lo stesso **serialVersionUID** durante la serializzazione degli oggetti scambiati via socket.

Per quanto concerne la persistenza dei dati, è stato utilizzato **SQLite**, motore DBMS leggero ma molto affidabile che garantisce prestazioni ottimali per il livello di scalabilità richiesto su cui si appoggia il **pattern DAO** da noi scelto di adottare. Tramite l'astrazione offerta dall'interfaccia *DAO* e dalle sue specifiche implementazioni (*UtenteDAO*, *PartitaDAO*, *AdminDAO (Server)*), il livello di business (es. *GestoreAutenticazione*) è totalmente disaccoppiato dalle specifiche query SQL rendendo il sistema flessibile a eventuali cambi di DBMS futuri.

Principali Scelte Progettuali

Durante lo sviluppo del sistema, sono state prese diverse decisioni architetturali e tecniche per soddisfare i requisiti funzionali e non funzionali richiesti.

Protocollo di Rete e Scambio dati:

Per la comunicazione Client-Server si è scelto di utilizzare le Socket TCP per garantire l'affidabilità della consegna dei pacchetti. Invece di scambiare stringhe testuali formattate, si è optato per la Serializzazione di Oggetti Java tramite `ObjectInputStream` e `ObjectOutputStream`. È stato definito un protocollo applicativo custom basato sulla classe `Messaggio`, che incapsula un identificatore dell'operazione (Enum `TipoMessaggio`) e un payload generico (`Object`), permettendo di gestire con un'unica interfaccia flussi diversi come Login, Mosse di gioco ed Errori.

Concorrenza e Asincronia

Il sistema fa un uso intensivo del multithreading per evitare blocchi dell'esecuzione (blocking I/O):

- **Lato Server:** È stato implementato un modello *Thread-per-Client*. Per ogni giocatore connesso, il server delega la gestione del flusso di I/O a un `ClientHandler` dedicato che implementa l'interfaccia `Runnable`.
- **Analisi dei Documenti (Stream API):** Per soddisfare il requisito di elaborazione dei file testuali, l'analisi per il calcolo della *Term Frequency* è stata delegata a un sotto-thread tramite la classe JavaFX `Task`. L'algoritmo di conteggio fa uso massiccio delle **Stream API** di Java 8, che offrono un approccio funzionale ed efficiente all'elaborazione delle stringhe, garantendo al contempo che l'interfaccia grafica dell'Amministratore non subisca alcun congelamento (freeze) durante l'elaborazione di file di grandi dimensioni.

Sicurezza e Prevenzione dei Cheat

Seguendo il principio del *Server Autoritativo*, la logica di validazione della parola non è mai esposta al Client. Nell'invio dell'oggetto **Sfida** ai giocatori, informazioni sensibili come la **parolaOriginale** e lo **shift** del Cifrario di Cesare sono state dichiarate con la keyword **transient**. Questo accorgimento tecnico impedisce che tali variabili vengano serializzate e trasmesse sulla rete, precludendo a un utente malintenzionato la possibilità di intercettare la soluzione analizzando i pacchetti di rete.

Configurazione Flessibile e Build Process

Per evitare l'uso di parametri hardcoded (es. indirizzo IP, porta, credenziali di root), sono state implementate classi **ConfigManager** che leggono tali valori da file esterni **.properties**. Infine, per la compilazione e la distribuzione, è stato impiegato **Maven** abbinato al **maven-assembly-plugin**. Questo ha permesso di produrre eseguibili "Fat JAR", che incapsulano autonomamente sia il bytecode del progetto che tutte le dipendenze di terze parti (come il driver JDBC di SQLite), garantendo la portabilità del software (requisito indispensabile per la valutazione).

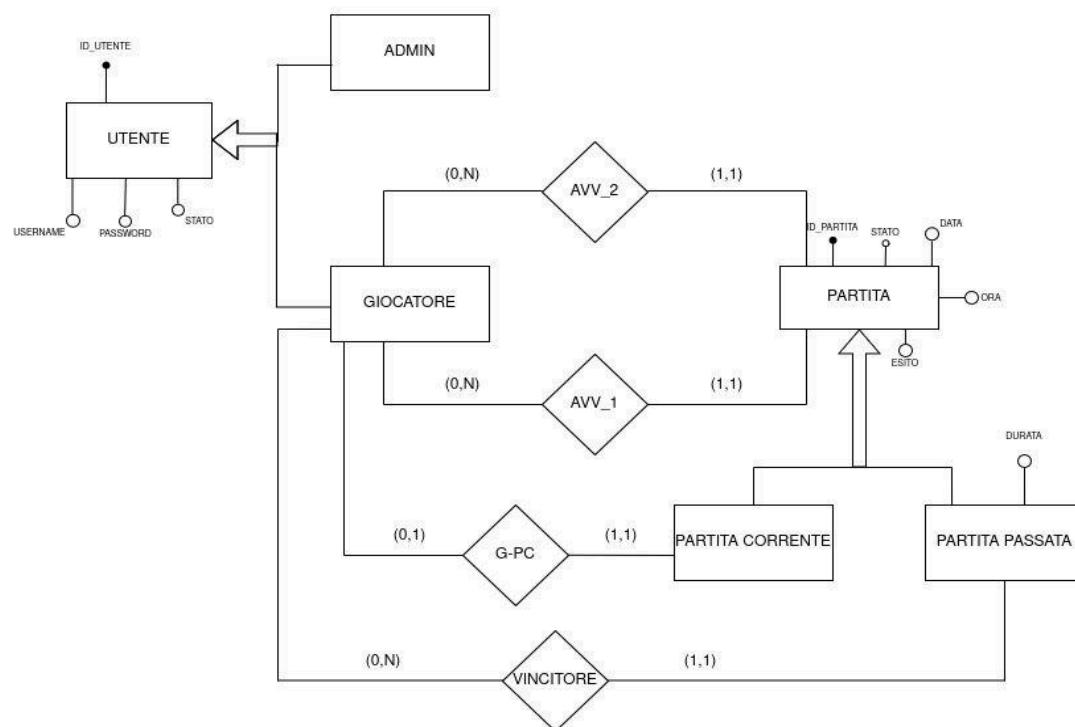
ID_UTENTE come chiave

Come principio di buona progettazione di un database, **USERNAME** dovrebbe essere l'identificatore primario. Per rendere aperto lo sviluppo e l'evoluzione del software, scegliere un id "artificiale" apre le porte a funzionalità aggiuntive future come ad esempio il cambio dello USERNAME (che altrimenti, con il modello *ideale*, avrebbe portato ad aggiornamenti a cascata del database, causando grandi rallentamenti e disguidi per l'utenza del sistema).

Progettazione DB

Progettazione concettuale

Modello E-R



Dizionario dei dati

Il diagramma E-R tiene conto delle seguenti entità:

Entità	Descrizione	Attributi	Identificatore
Utente	Individuo generico registrato dall'applicativo, di cui si desidera conservare in modo persistente le credenziali di accesso necessarie all'autenticazione.	ID_Utente, Username, Password, Stato	ID_Utente
Admin	Utente che ha accesso a tutte le informazioni su giocatori e partite.	ID_Utente, Username, Password, Stato	ID_Utente
Giocatore	Utente che partecipa ad una partita contro un altro giocatore	ID_Utente, Username, Password, Stato	ID_Utente
Partita	Sessione di gioco tra due giocatori	ID_Partita, Stato, Data, Ora, Esito	ID_Partita
Partita_Passata	Partita registrata e terminata	ID_Partita, Stato, Data, Ora, Esito, Durata	ID_Partita
Partita_Corrente	Partita in corso	ID_Partita, Stato, Data, Ora, Esito	ID_Partita

Il diagramma E-R tiene conto delle seguenti associazioni:

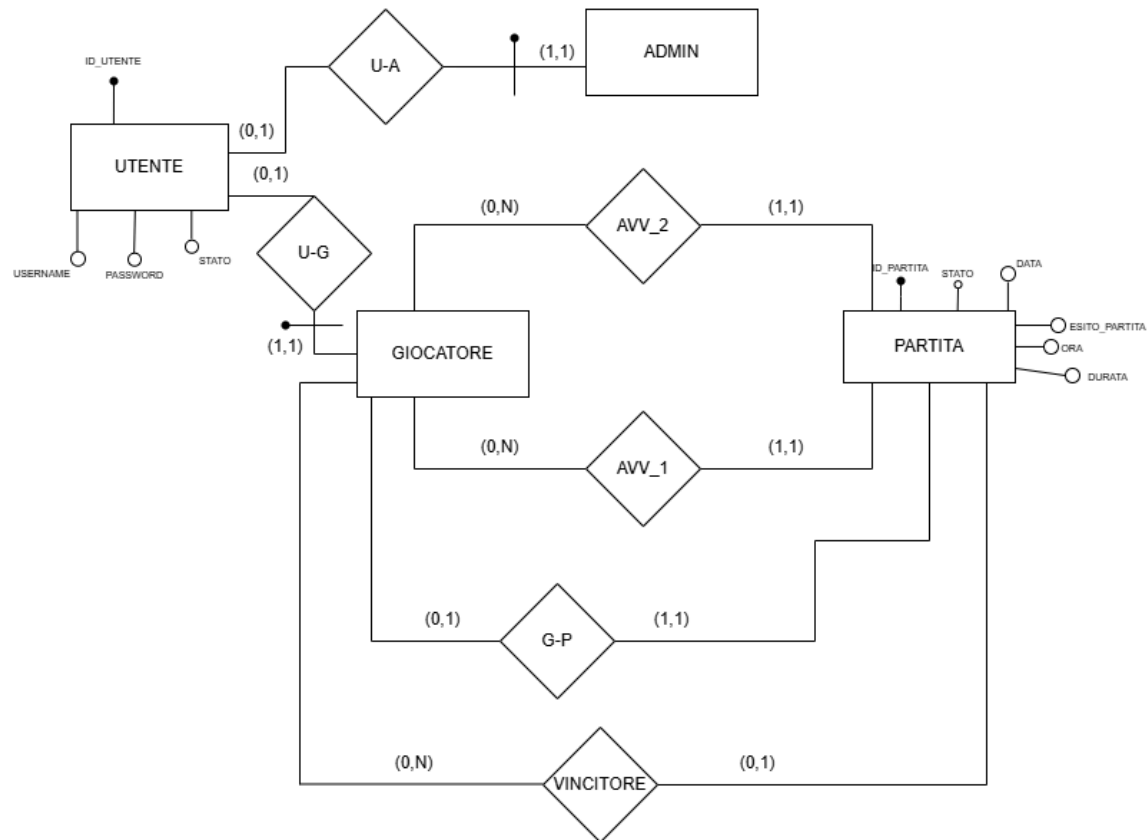
Associazione	Descrizione	Entità coinvolte	Attributi
Avv_1	Relationship che associa il giocatore 1 alla partita	Giocatore (0,N) Partita (1,1)	Nessuno
Avv_2	Relationship che associa il giocatore 2 alla partita	Giocatore (0,N) Partita (1,1)	Nessuno
G-PC	Relationship che associa un giocatore alla partita a cui sta partecipando	Giocatore (0,1) Partita_Corrente (1,1)	Nessuno
Vincitore	Relationship che indica il vincitore della partita terminata	Giocatore (0,N) Partita_Passata (1,1)	Nessuno

Vincoli non esprimibili tramite lo schema E-R:

- A una partita possono partecipare solo due giocatori a volta
- La durata di una partita passata è espressa in minuti
- In qualsiasi istante ci può essere una sola partita in corso
- Una partita ha inizio solo se entrambi i giocatori sono connessi e autenticati
- Il vincitore è il giocatore che da più in fretta la soluzione

Progettazione Logica

Ristrutturazione del modello E-R



La generalizzazione dell'entità Utente è stata eliminata sostituendola con due associazioni uno a uno: U-G, che si instaura tra le entità Utente e Giocatore, e U-A che si instaura tra le entità Utente e Admin.

Le entità Giocatore e Admin necessitano, come identificatore esterno, dell'attributo ID_Utente dell'entità Utente.

La generalizzazione dell'entità Partita, invece, è stata eliminata accorpendo le entità figlie nell'entità genitore: infatti l'entità Partita ora possiede l'attributo Durata ed è associata con l'entità Giocatore.

Traduzione

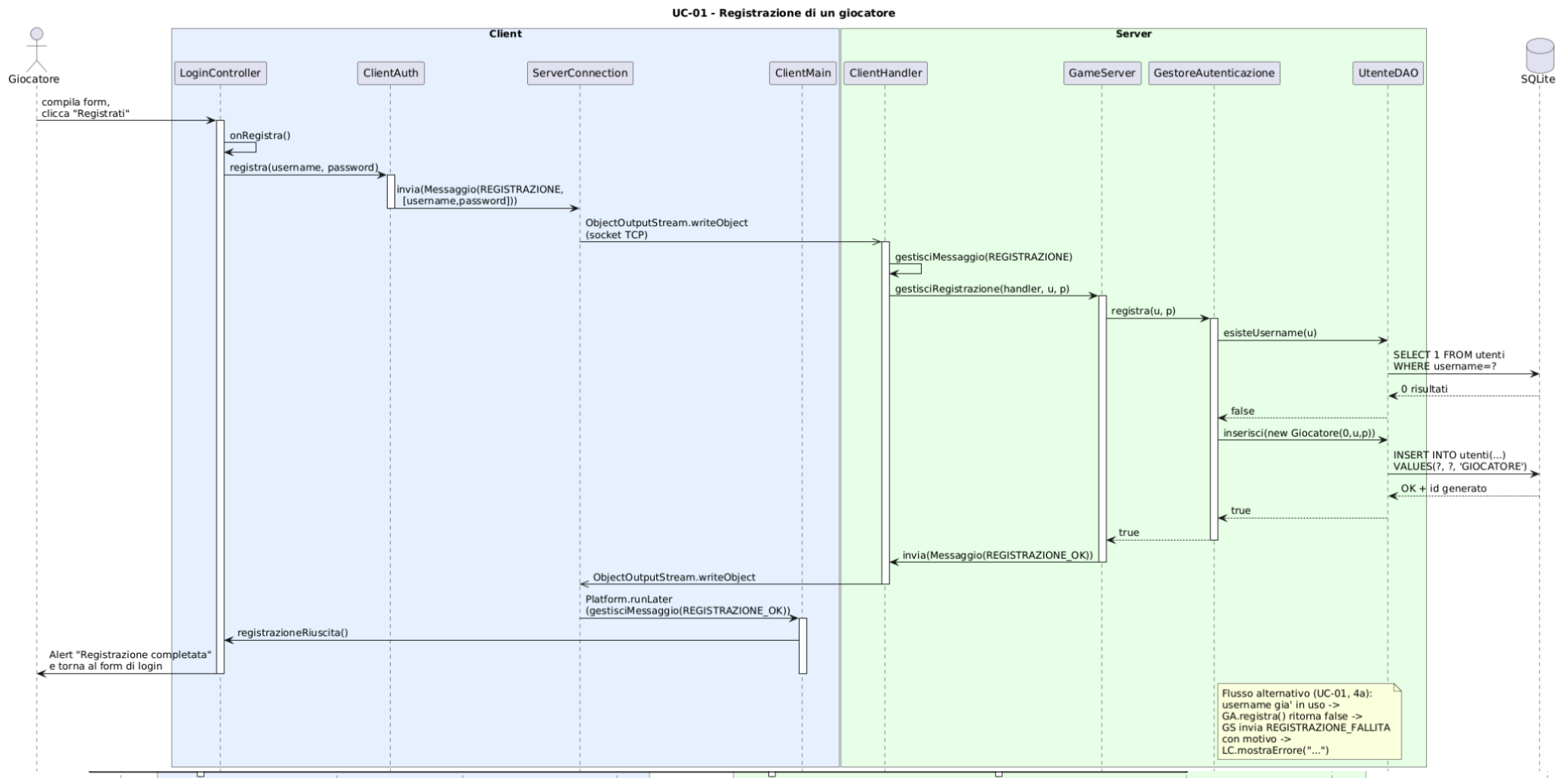
Utente(ID_Utente, Username, Password, Stato)

Admin(ID_Admin_FK)

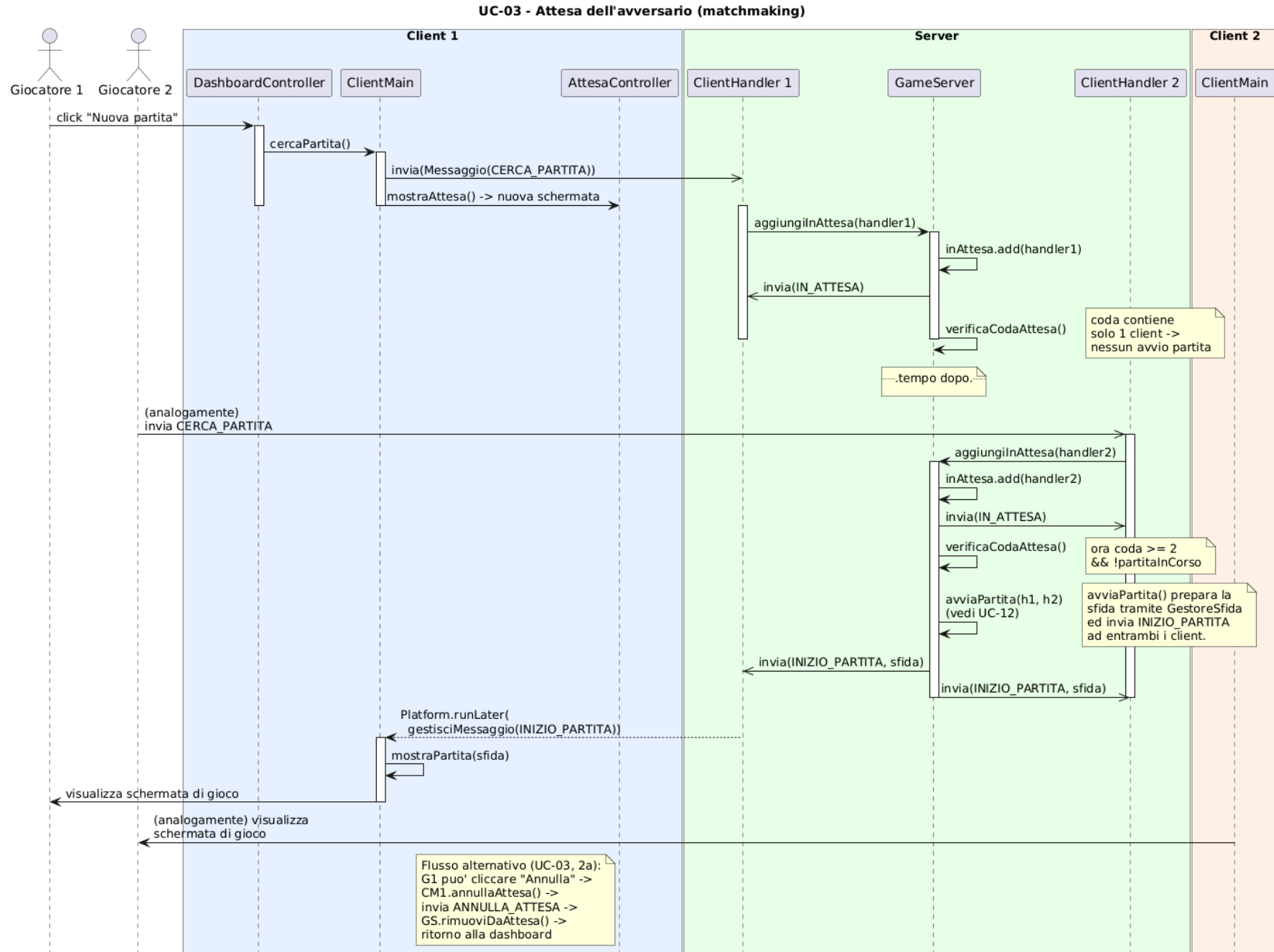
Giocatore(ID_Giocatore_FK)

Partita(ID_Partita, ID_Avversario1_FK, ID_Avversario2_FK, Data, Ora, Durata, Stato, ID_Vincitore_FK)

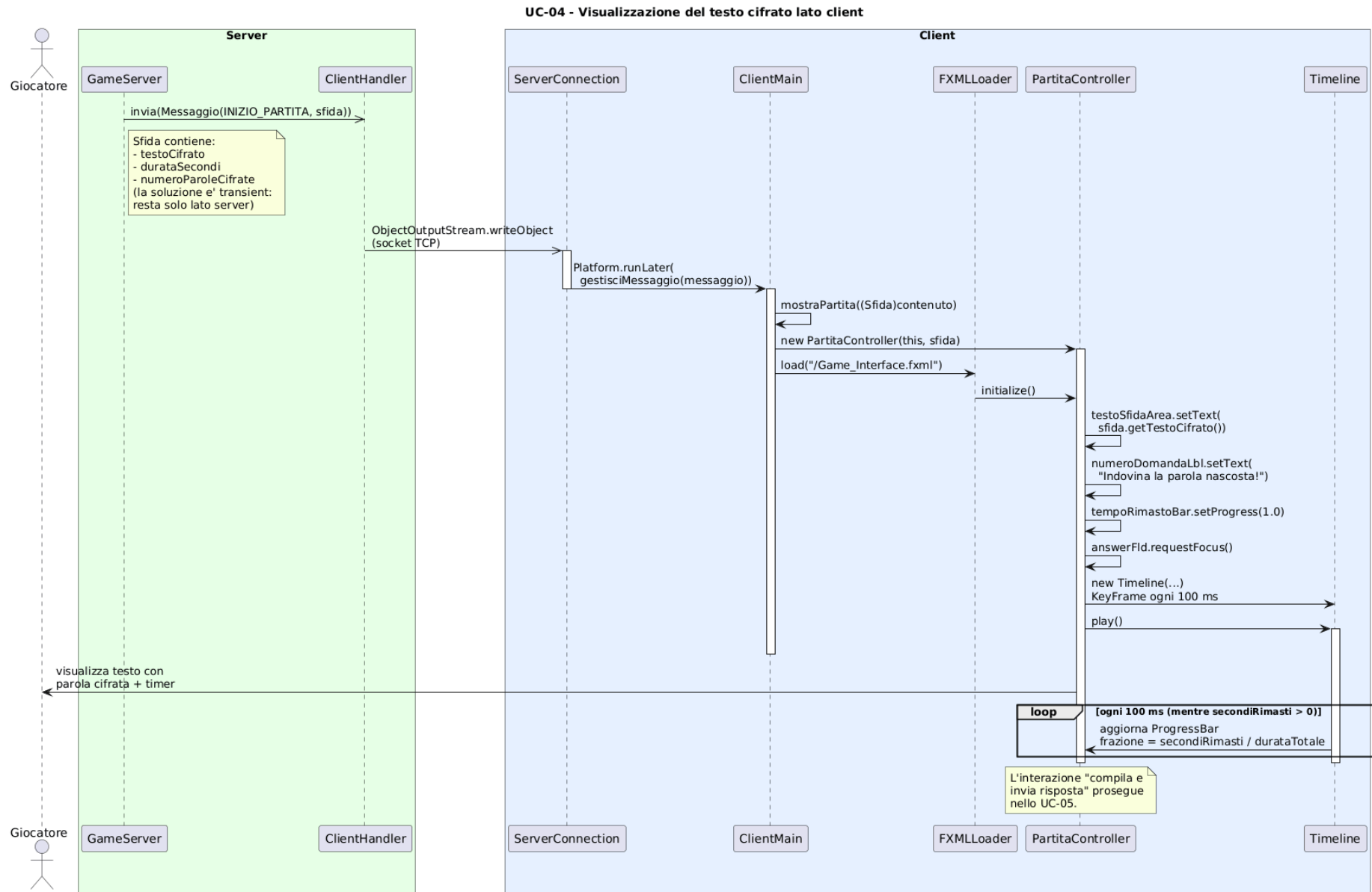
Diagrammi di sequenza



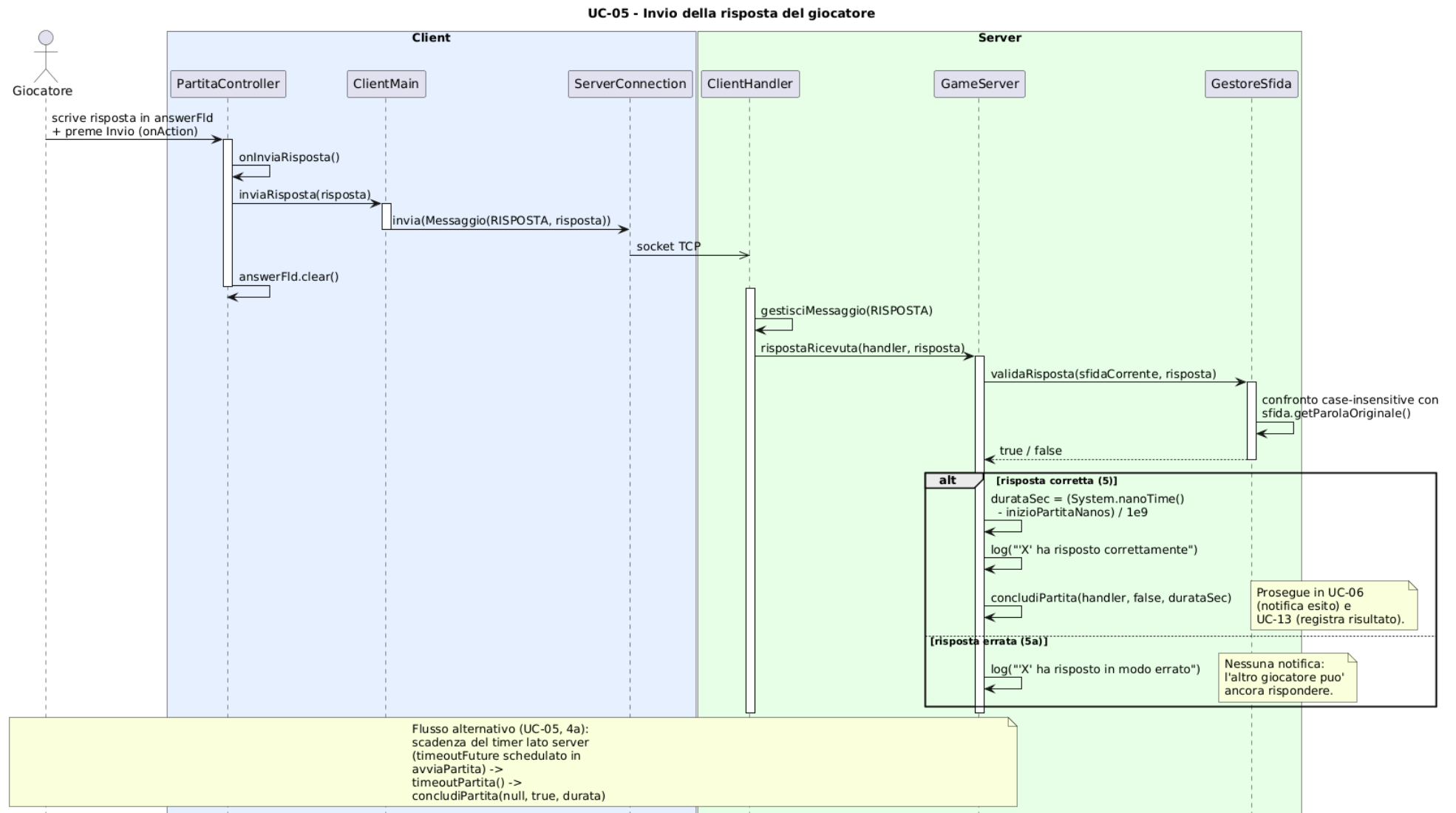
Diagrammi di sequenza



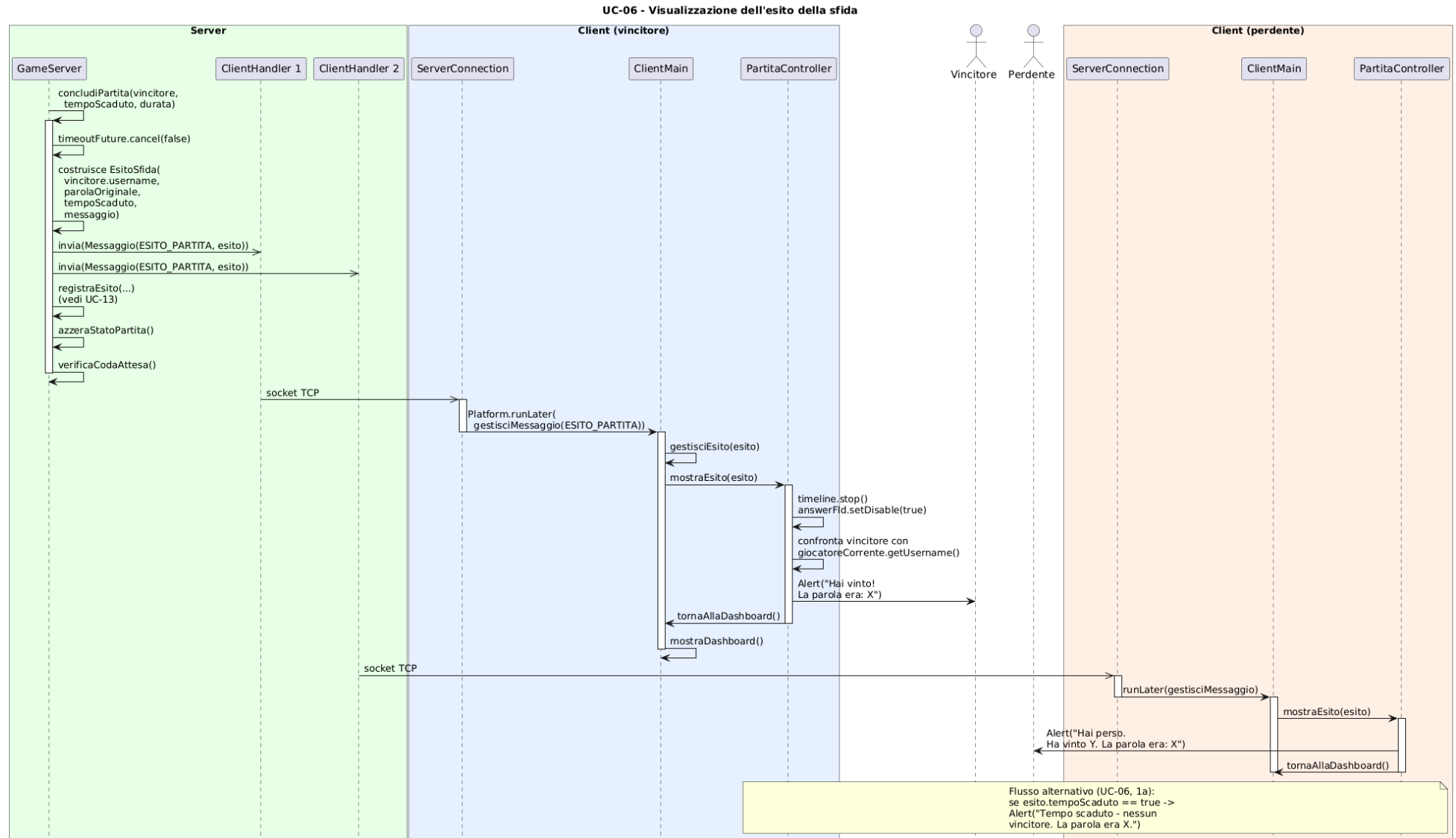
Diagrammi di sequenza



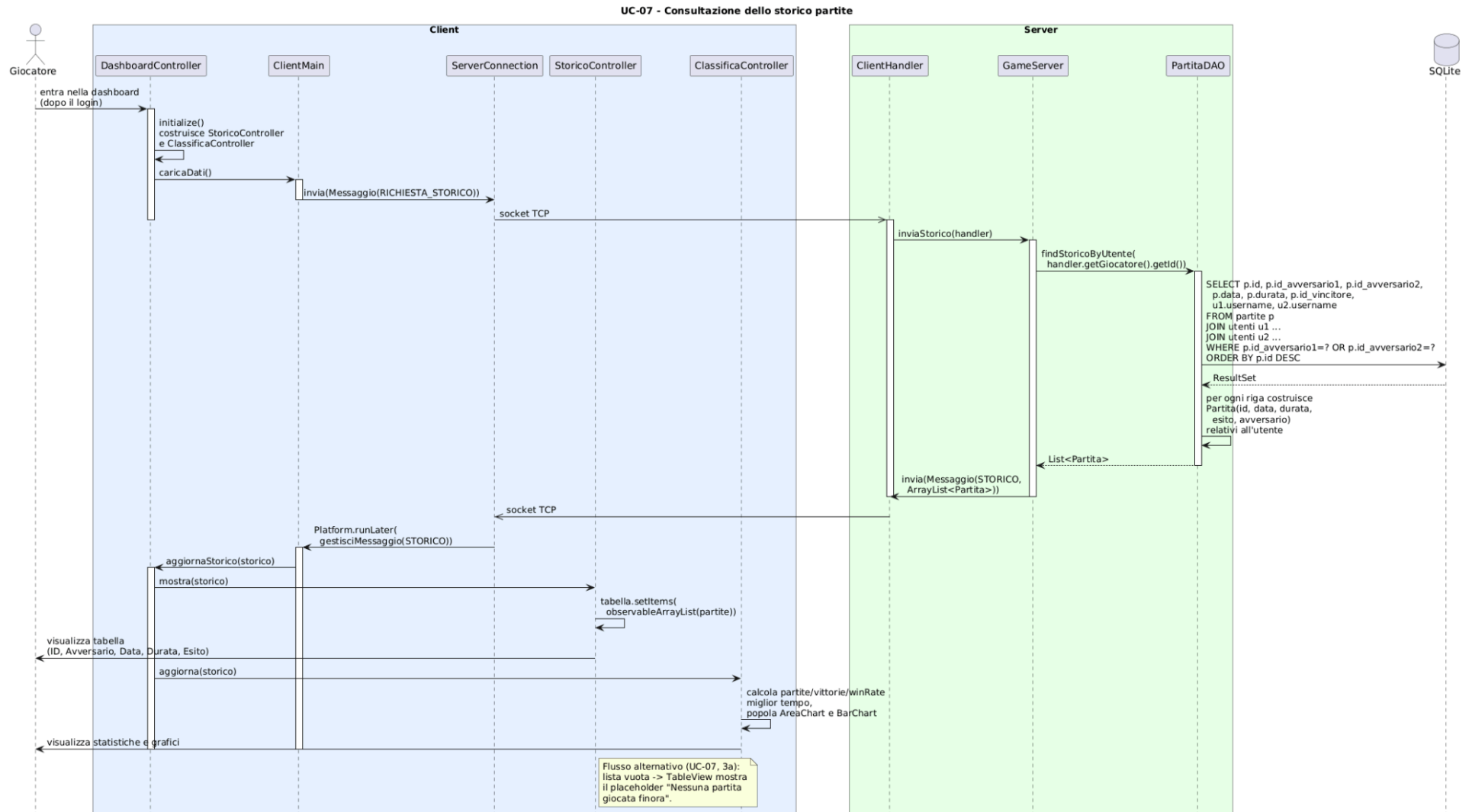
Diagrammi di sequenza



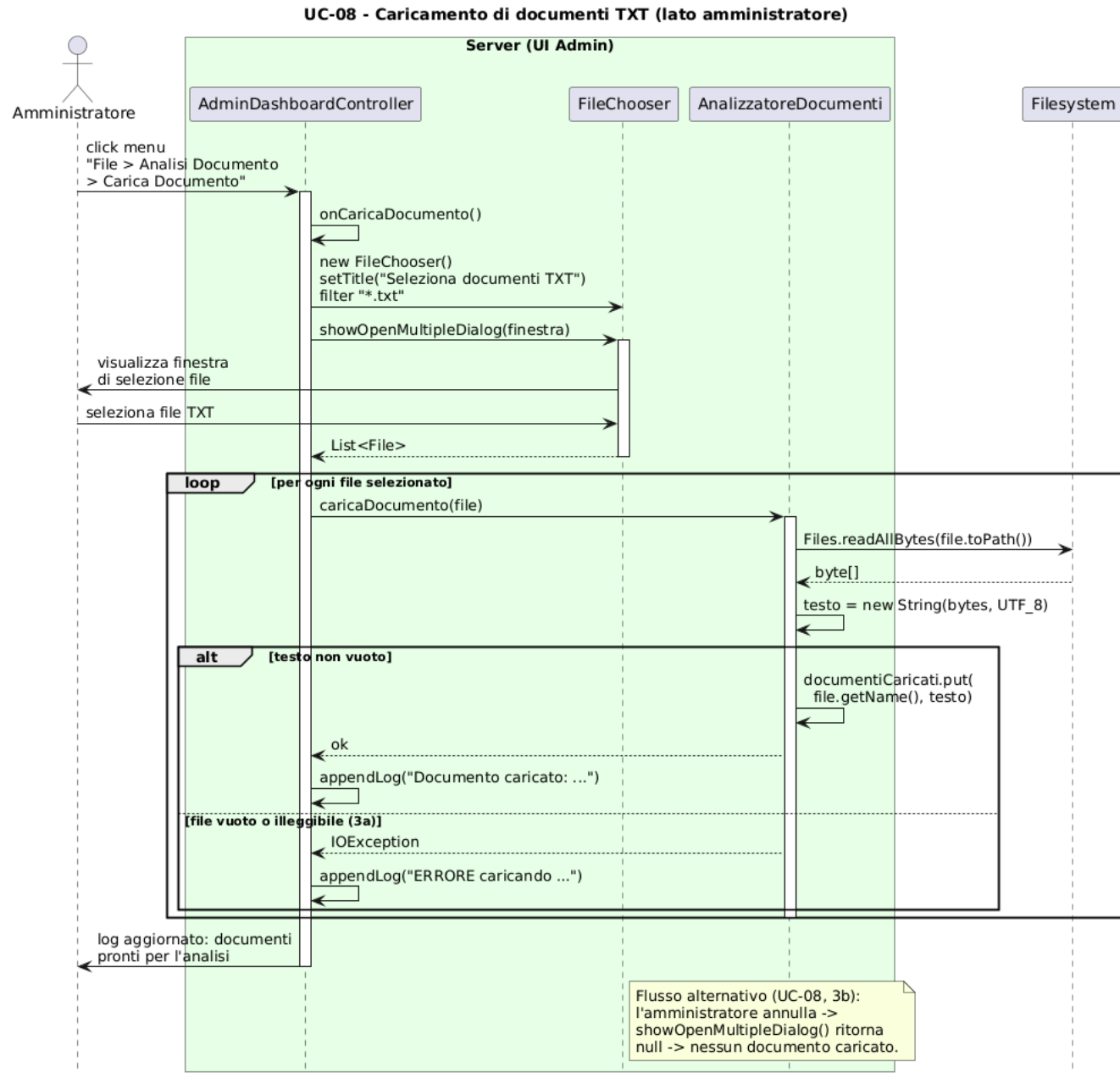
Diagrammi di sequenza



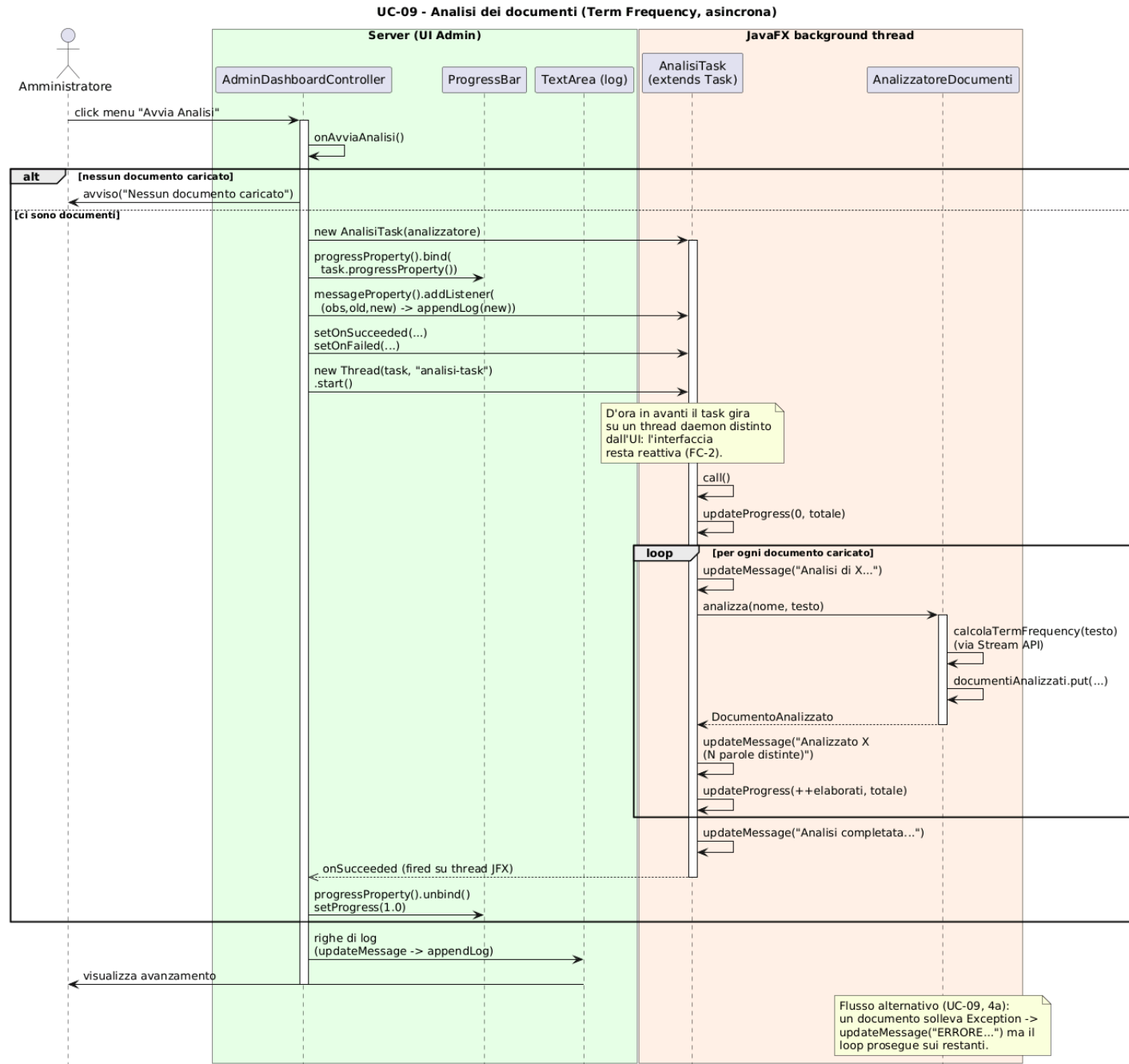
Diagrammi di sequenza



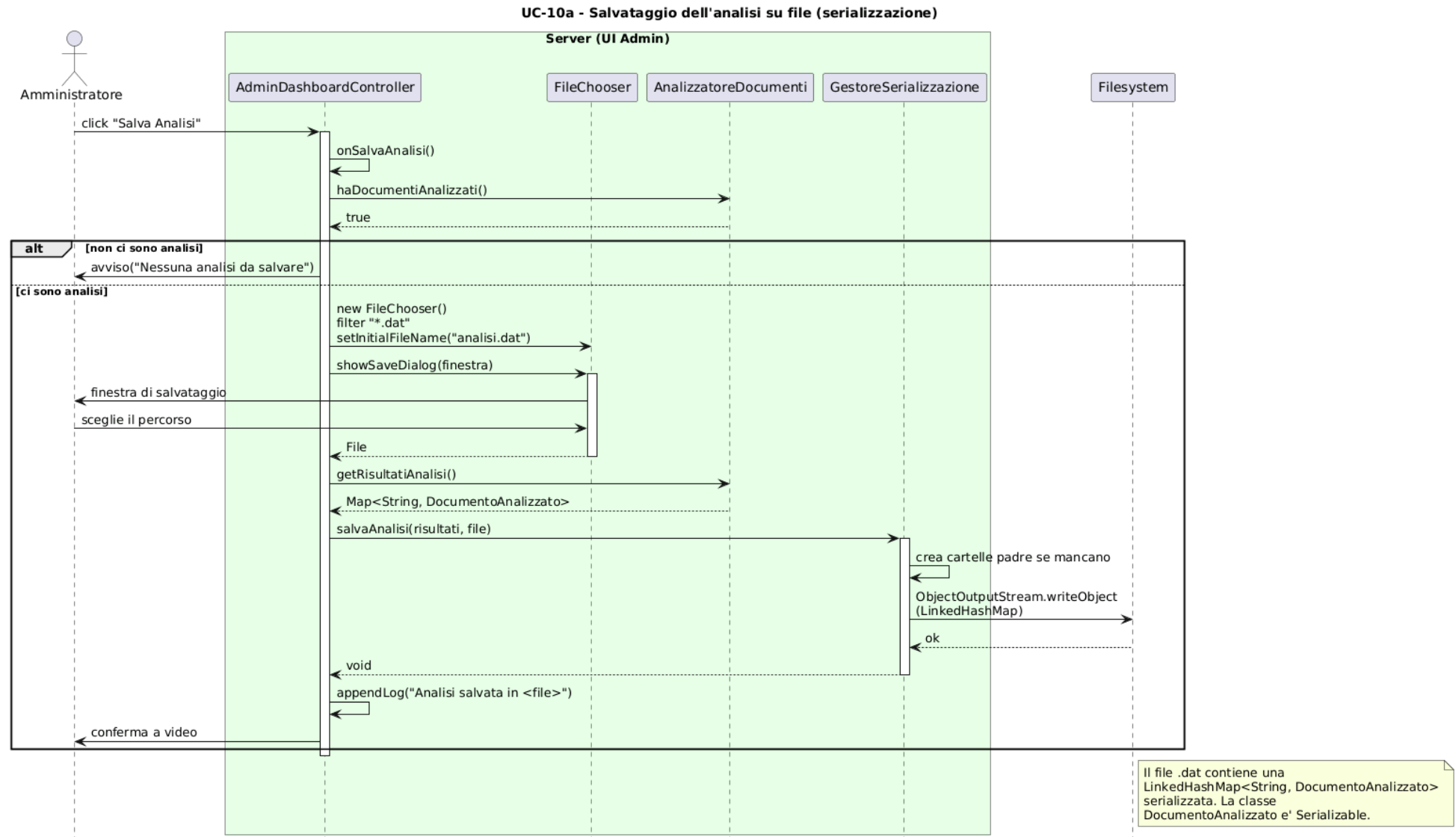
Diagrammi di sequenza



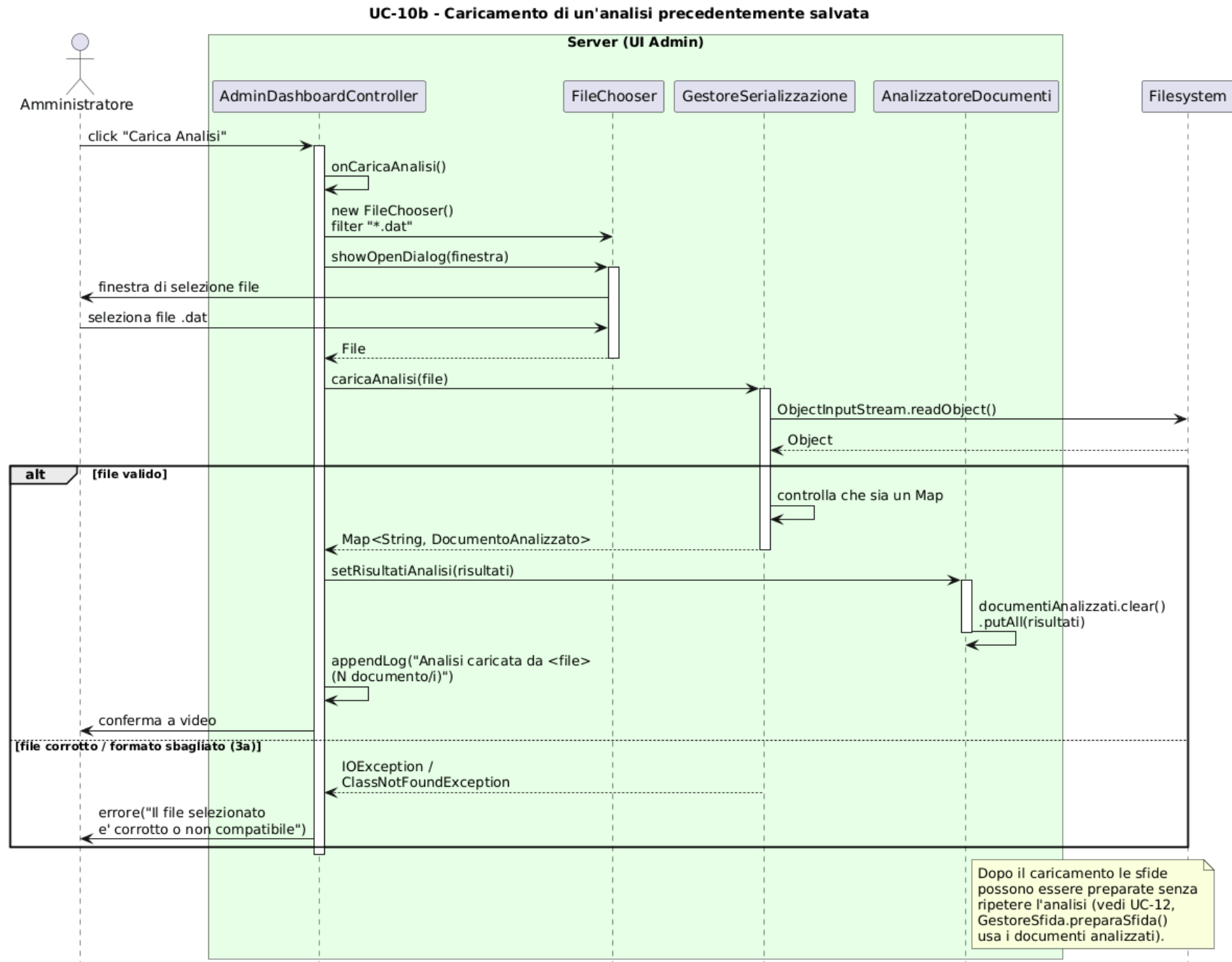
Diagrammi di sequenza



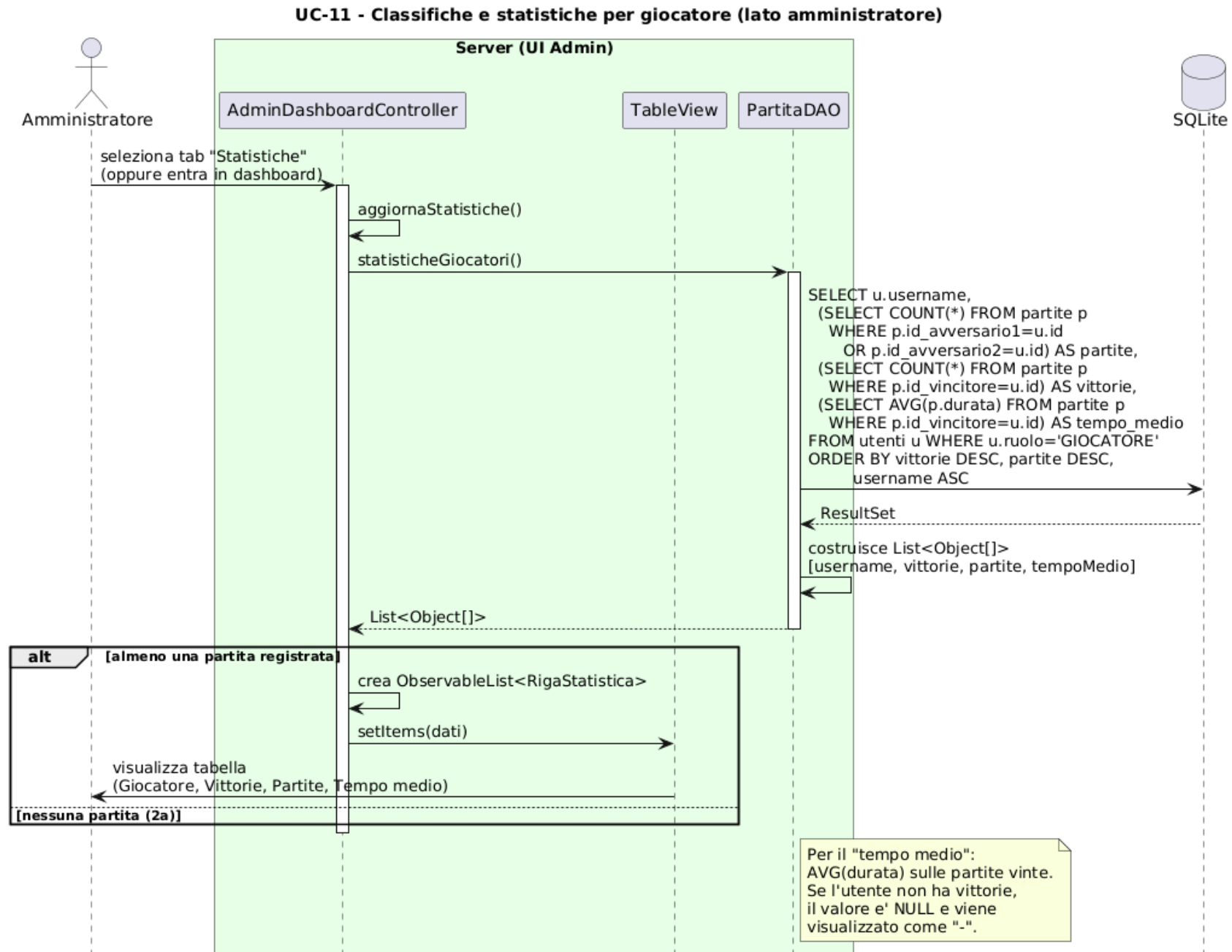
Diagrammi di sequenza



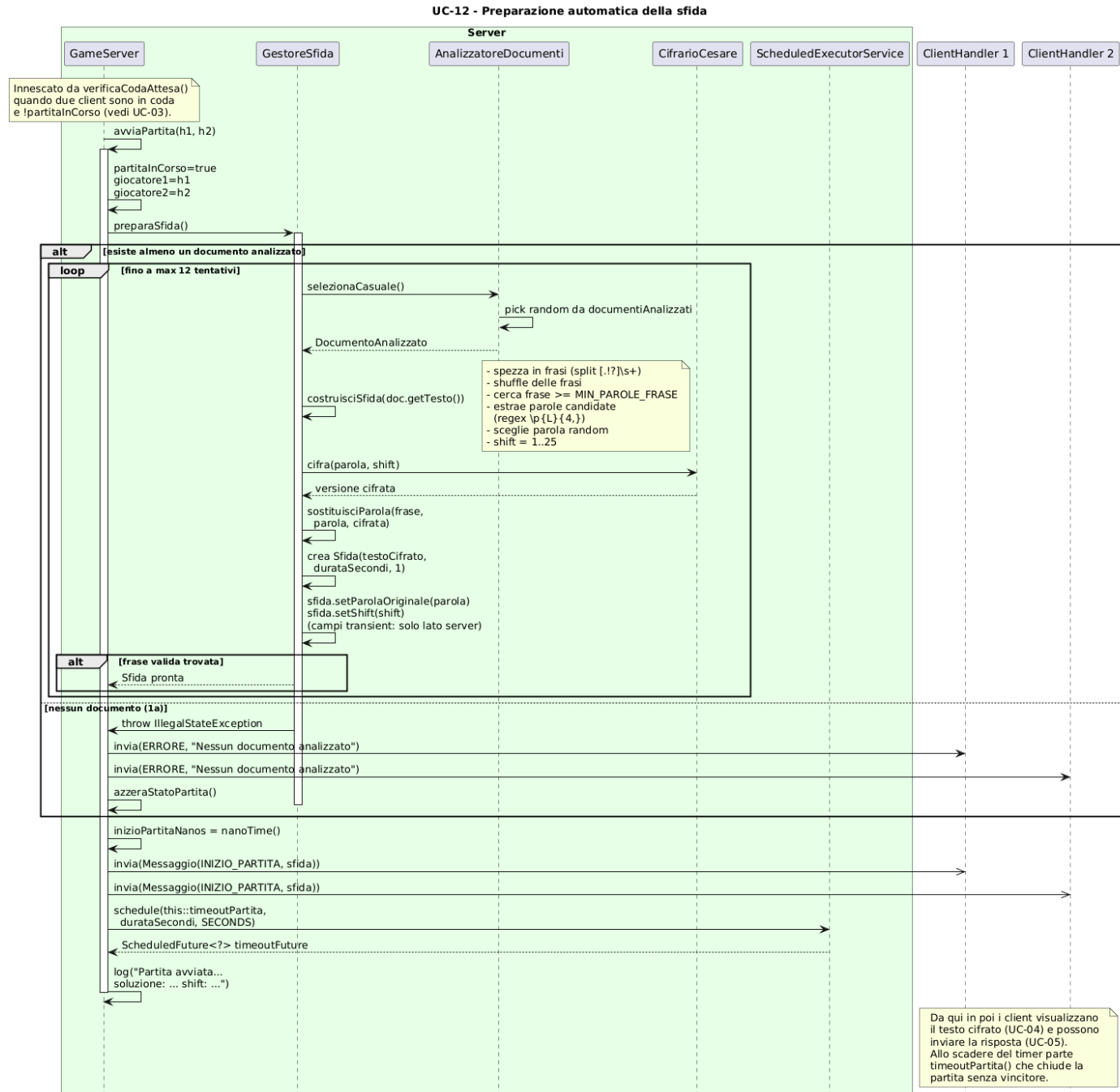
Diagrammi di sequenza



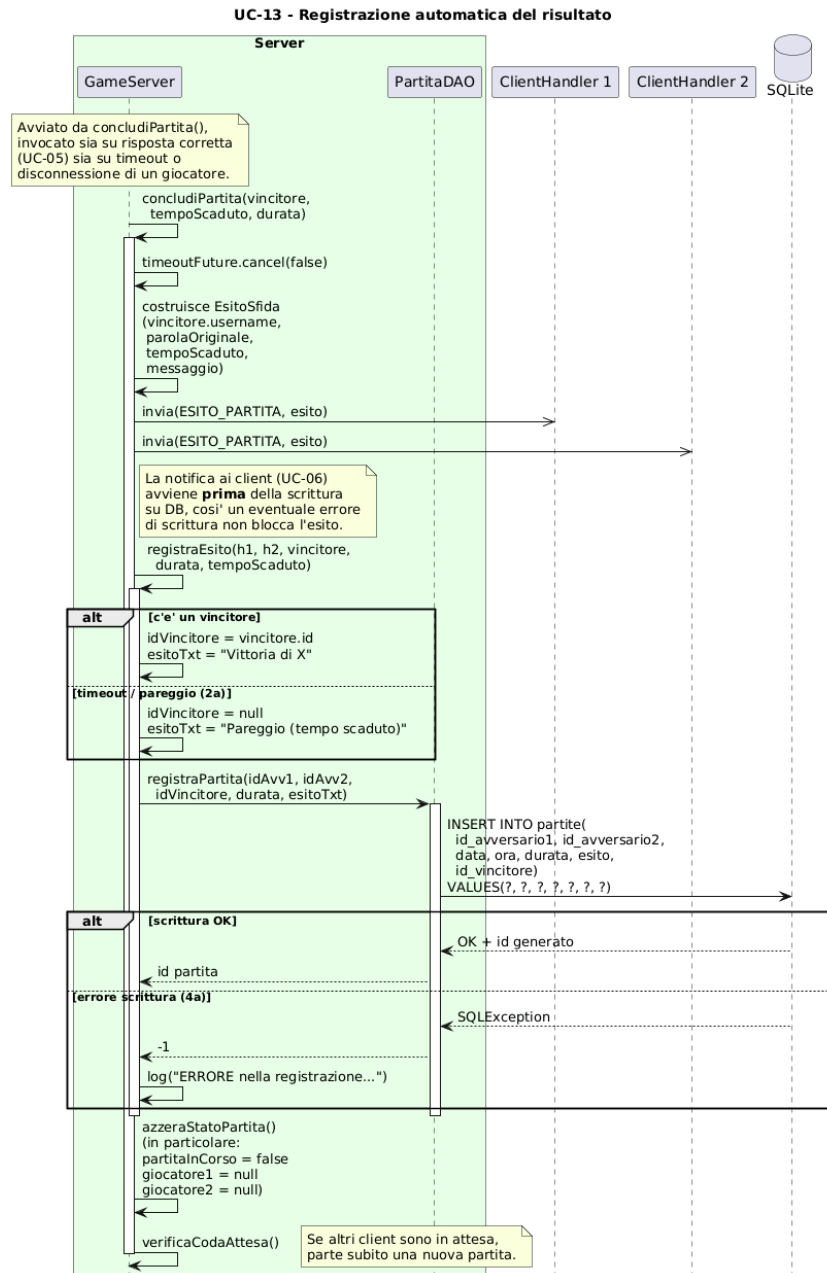
Diagrammi di sequenza



Diagrammi di sequenza



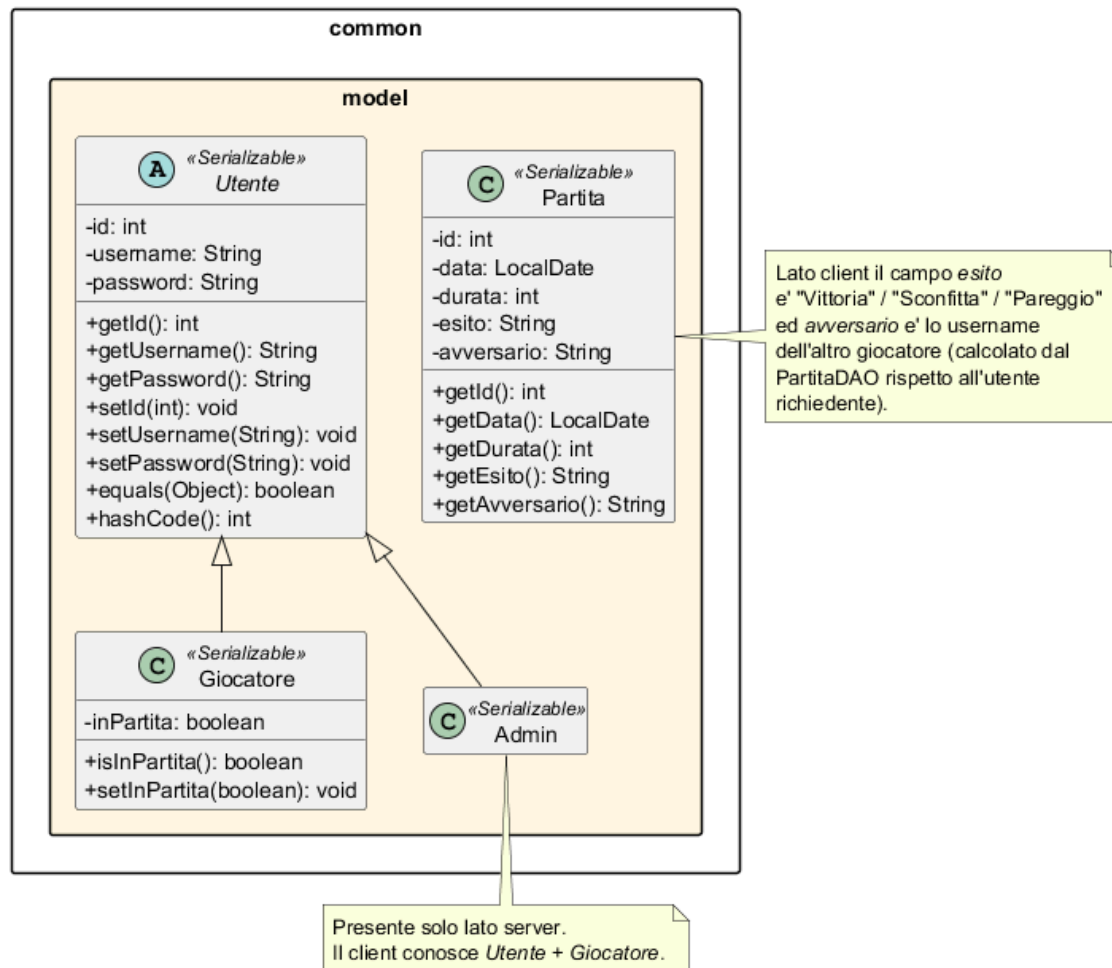
Diagrammi di sequenza



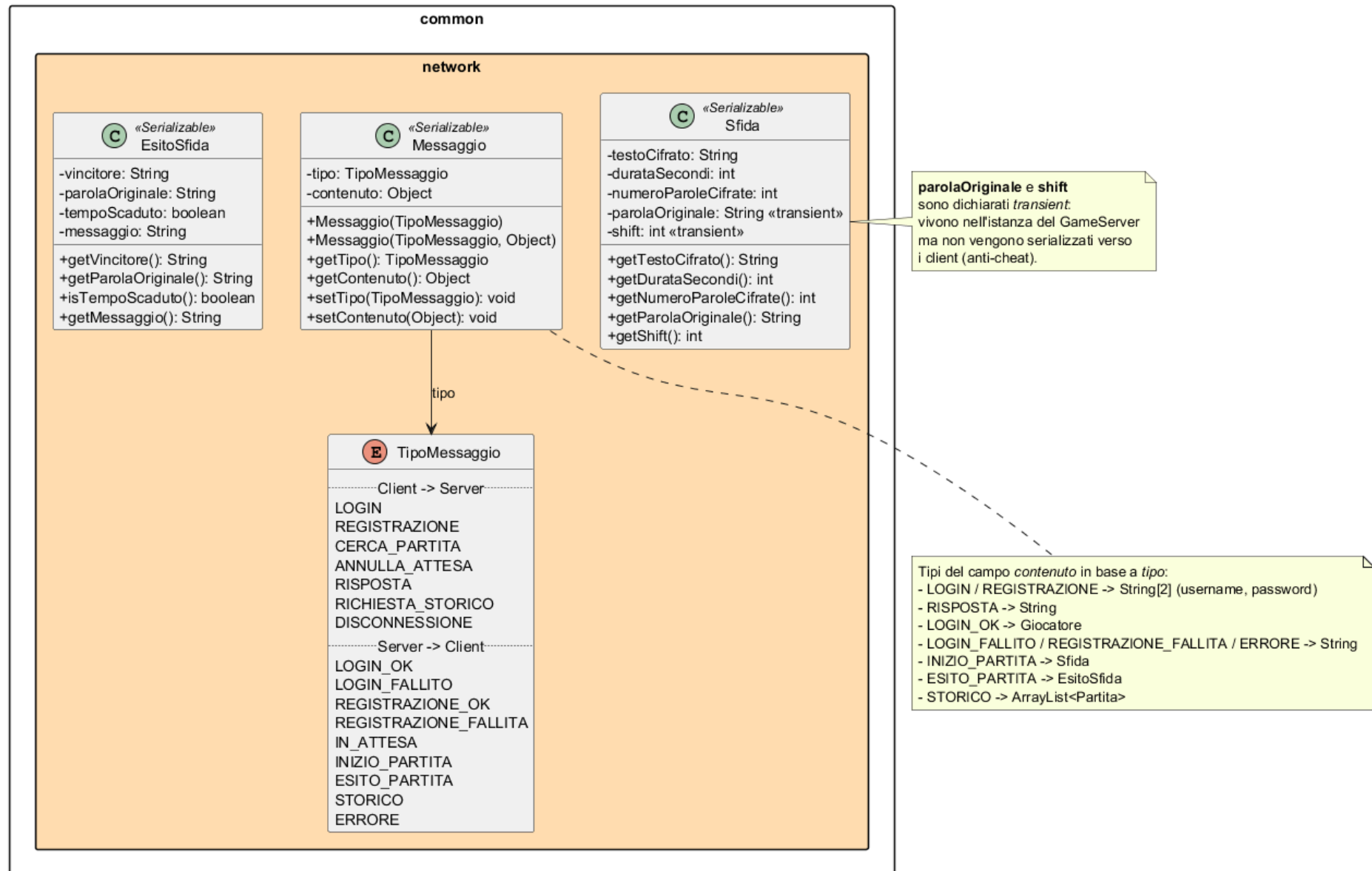
Diagrammi delle Classi

Moduli Client e Server

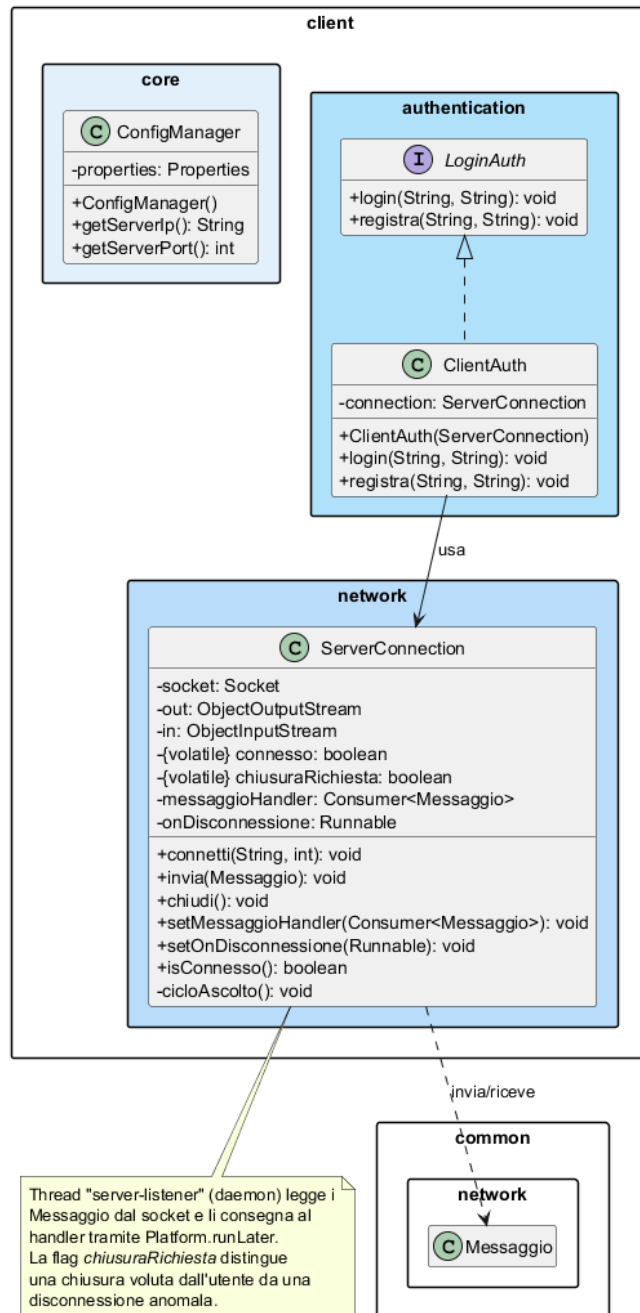
1. Modello condiviso - it.guesstheword.common.model



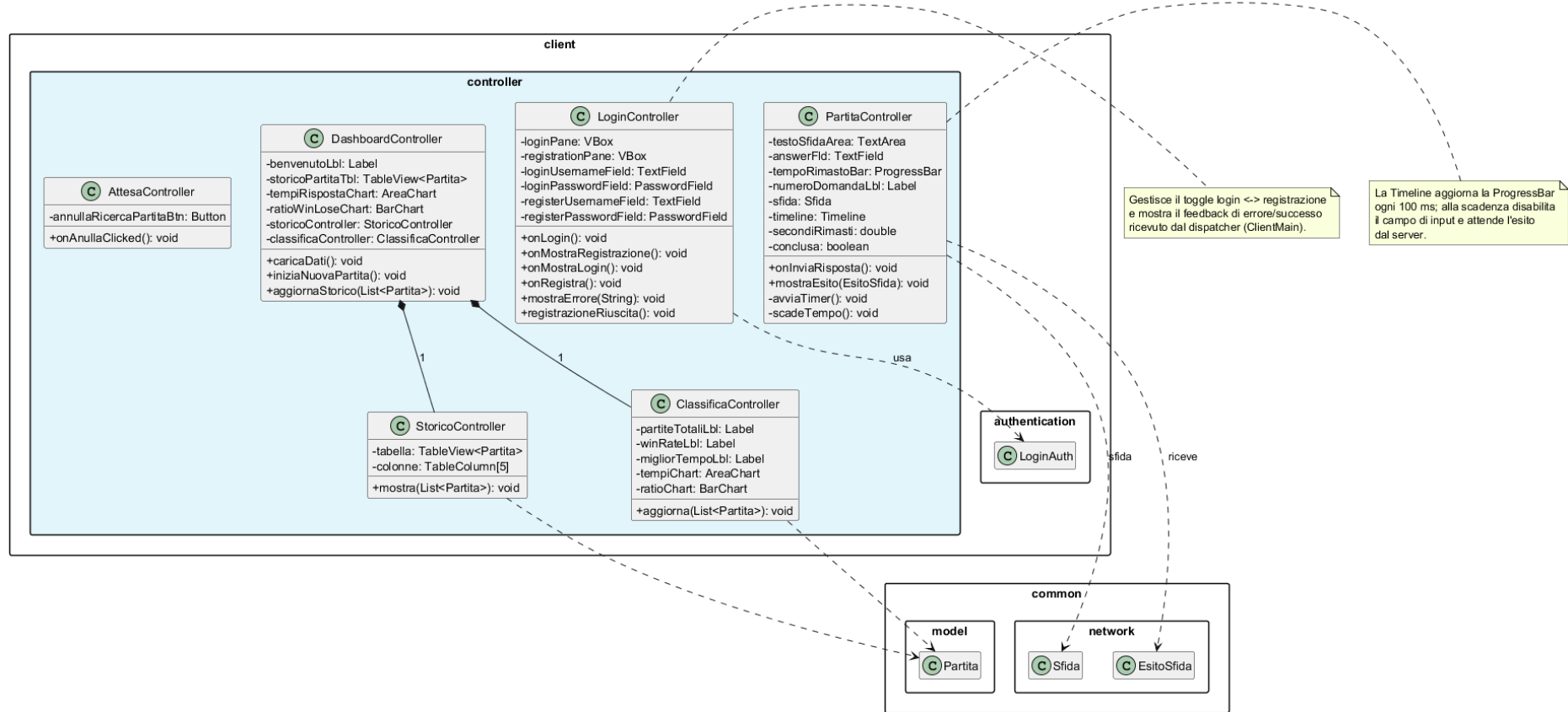
2. Protocollo condiviso - it.guesstheword.common.network



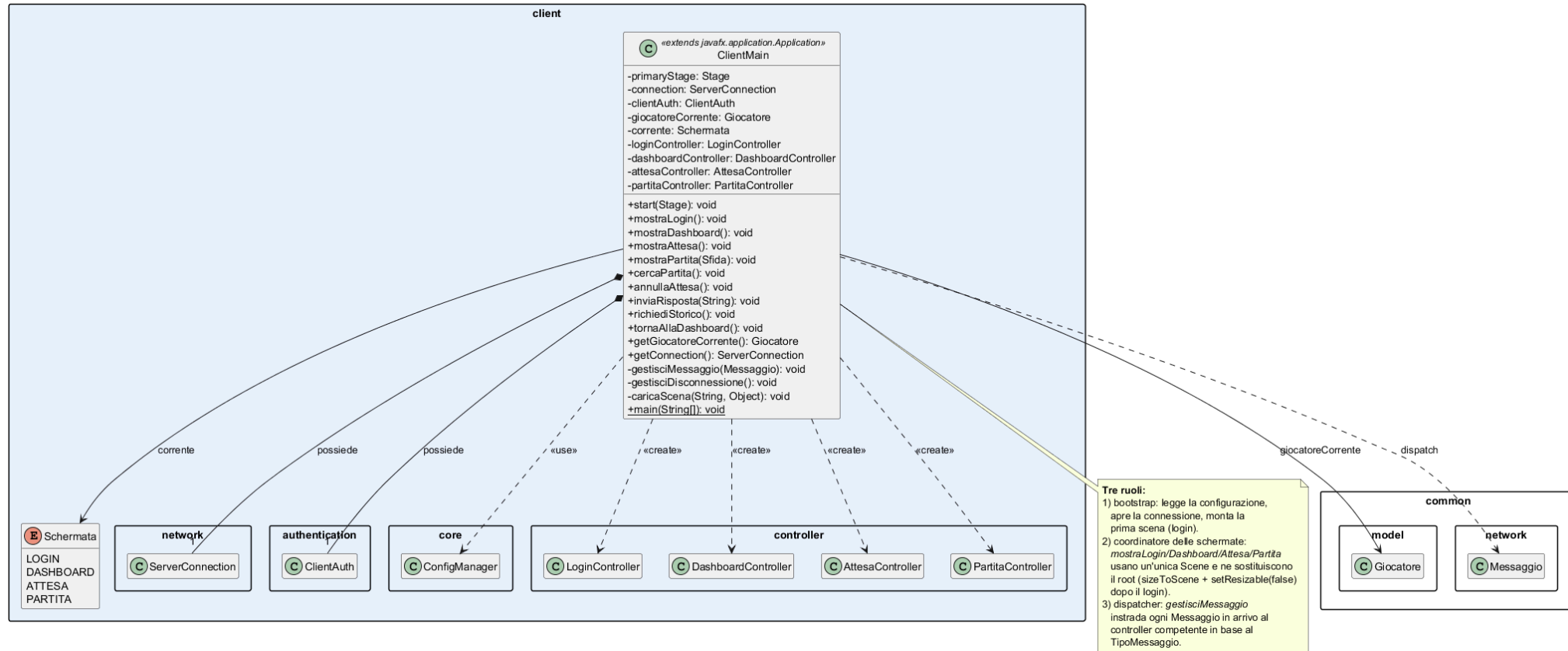
3. Client - Networking e autenticazione



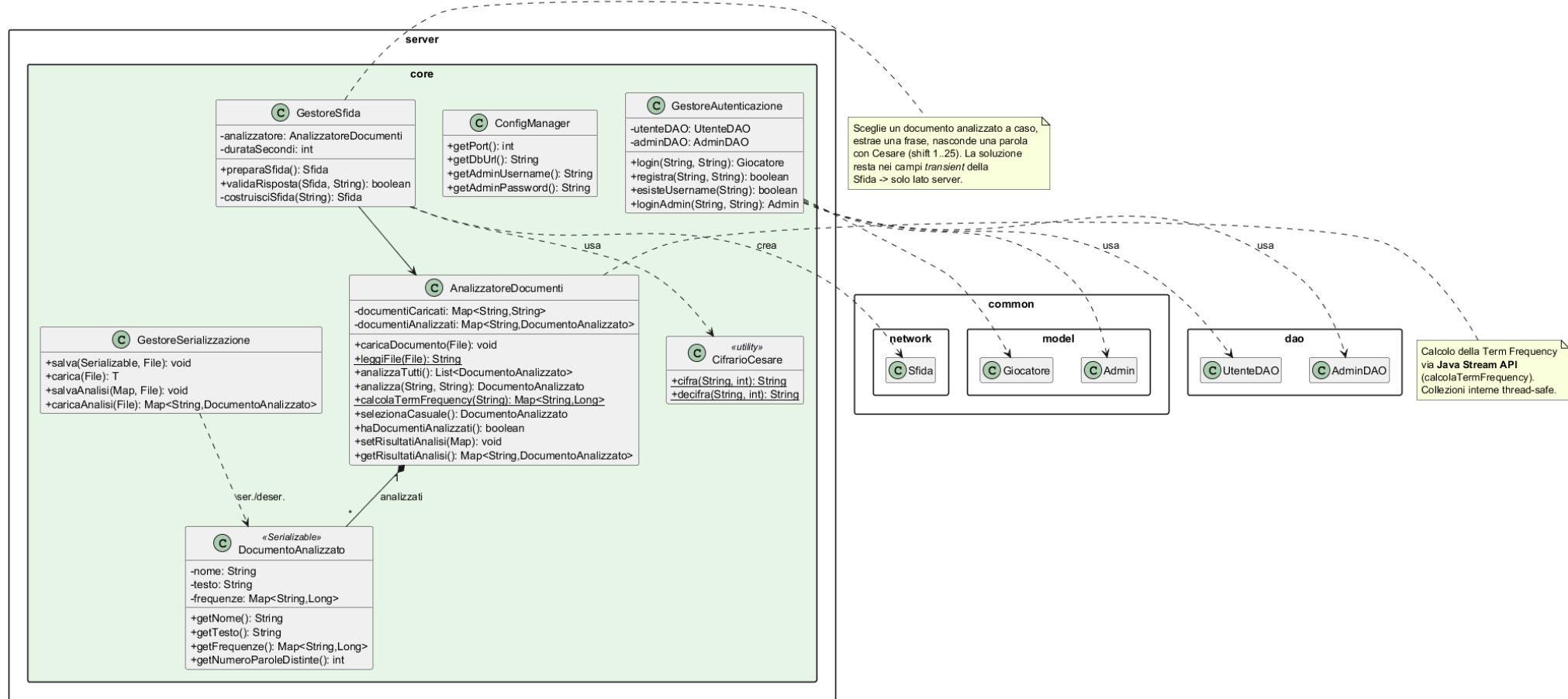
4. Client - Controller (it.guesstheword.client.controller)



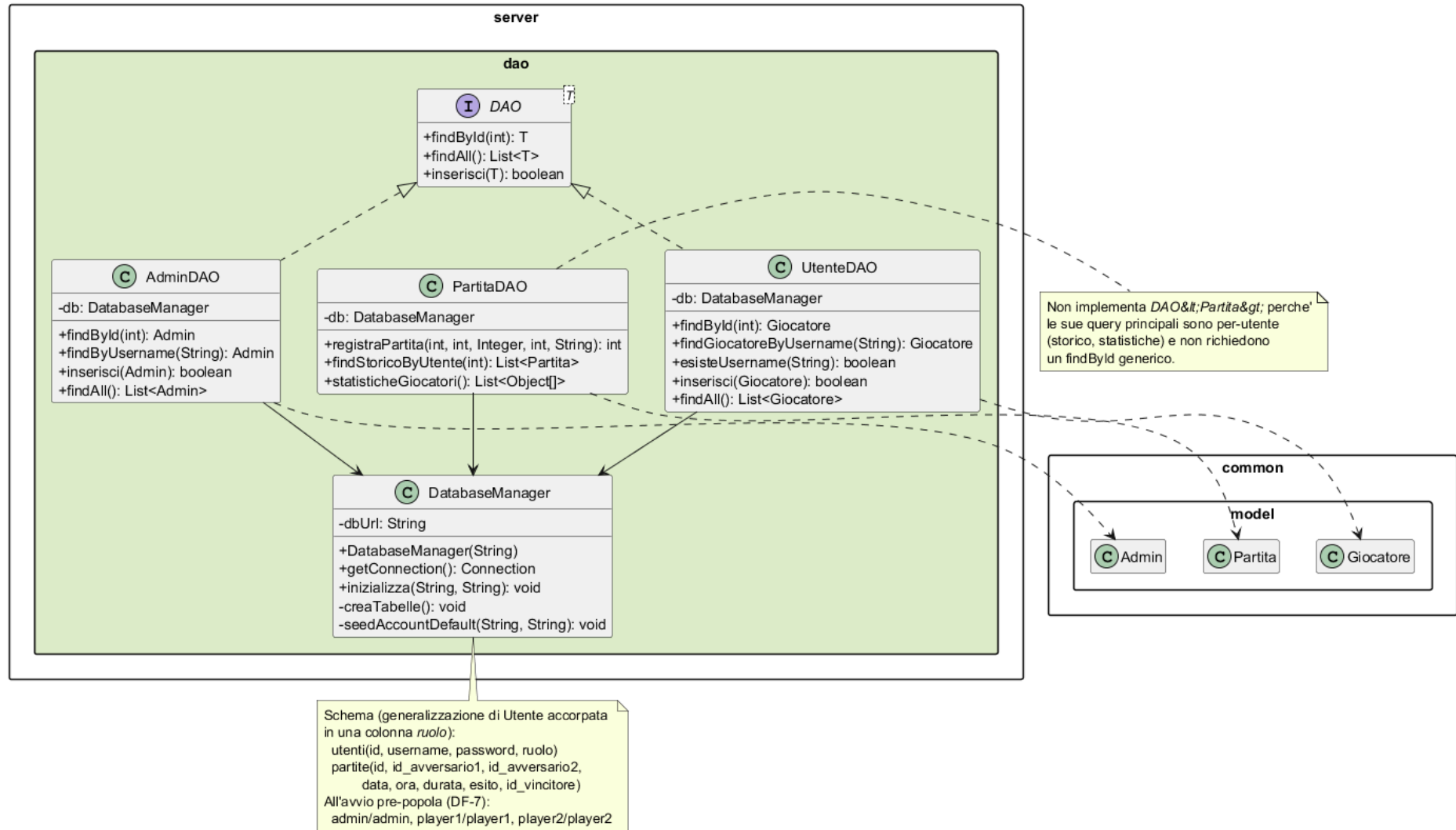
5. Client - Bootstrap e dispatcher (ClientMain)



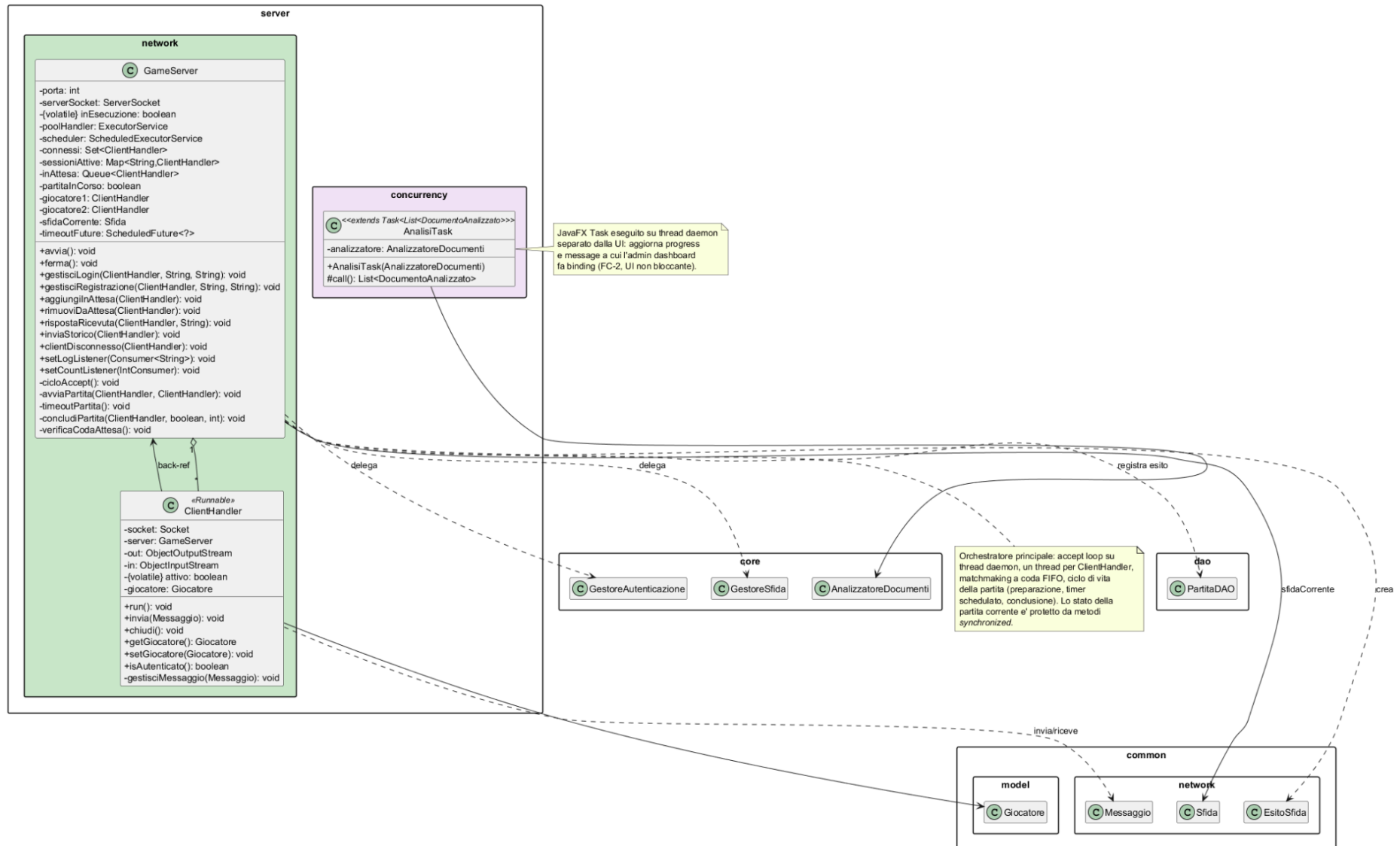
6. Server - Core (it.guesstheword.server.core)



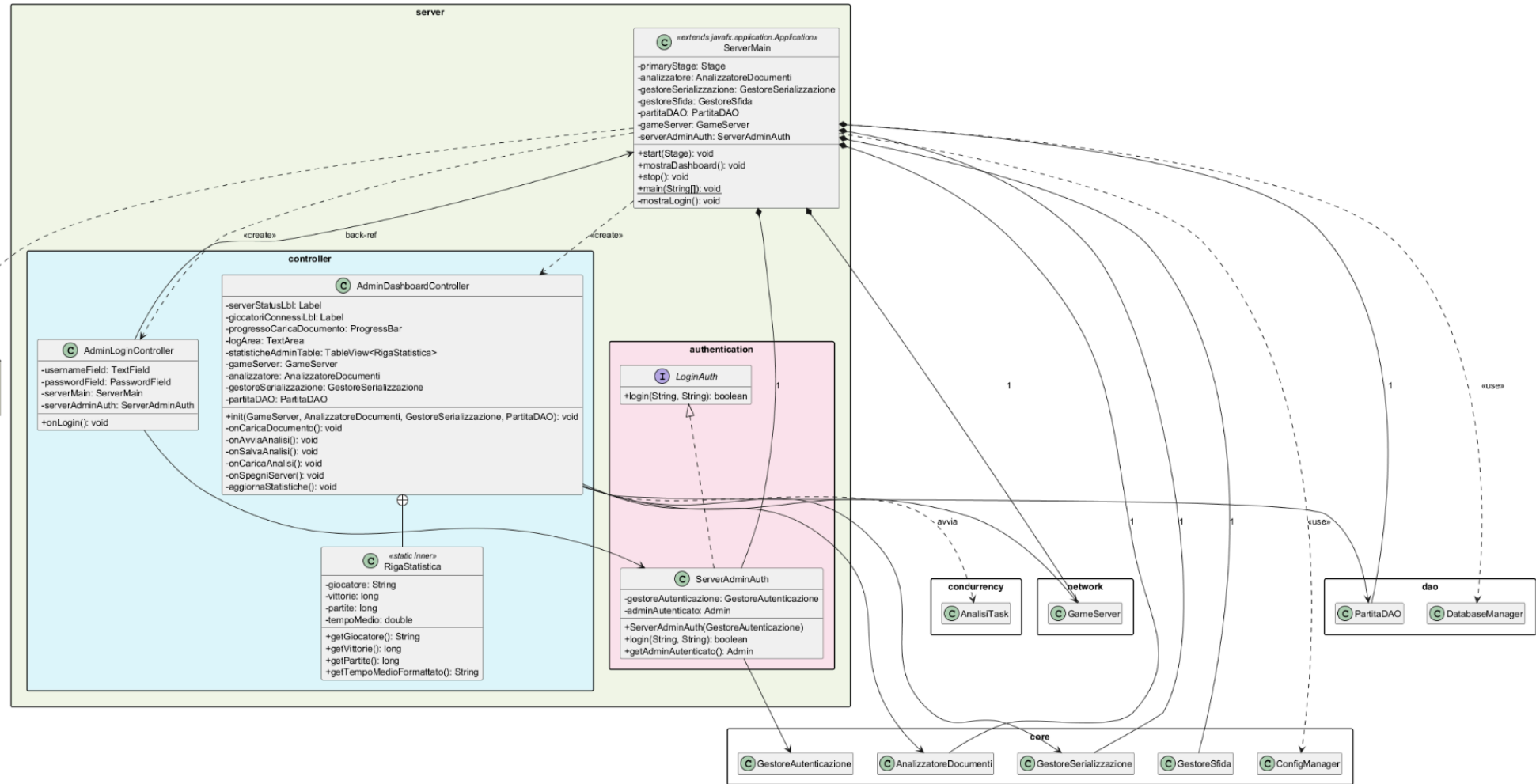
7. Server - Persistenza (it.guesstheword.server.dao)



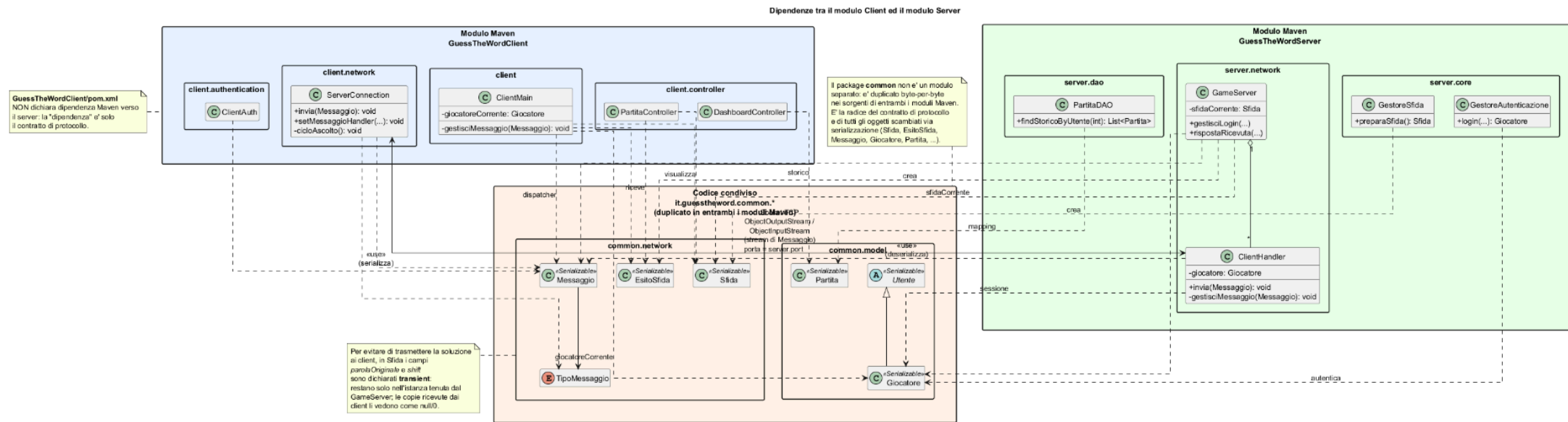
8. Server - Networking e concorrenza



9. Server - Bootstrap, autenticazione admin e controller



Dipendenze Intermodulari tra Client e Server



Conclusioni e Sviluppi Futuri

La fase di progettazione e architettura ha tradotto con successo i requisiti funzionali in una struttura software solida, modulare e ad alte prestazioni. L'adozione di un approccio multi-modulo tramite Apache Maven ha permesso una netta separazione delle responsabilità (Separation of Concerns), isolando la logica di rete del Server, le interfacce e i controller del Client, e gli oggetti condivisi (Common/DTO).

Dal punto di vista della rete e della concorrenza, l'implementazione di Socket TCP nativi, combinata con una gestione multi-threaded tramite componenti dedicati (come i ClientHandler lato server), garantisce la stabilità delle comunicazioni e la capacità di gestire una sessione di gioco tra due client in modo ottimale, prevenendo colli di bottiglia o condizioni di corsa (Race Conditions) nell'accesso alle risorse condivise. L'integrazione del livello di persistenza dei dati (Database) assicura inoltre l'integrità e la coerenza delle informazioni di gioco e dei profili utente.

Sebbene l'architettura attuale soddisfi pienamente i requisiti ingegneristici richiesti, il sistema è stato progettato seguendo i principi del clean code e dei design pattern, lasciando la strada aperta a future evoluzioni scalabili, tra cui:

1. **Sicurezza delle comunicazioni:** Transizione dai Socket TCP standard a connessioni cifrate tramite protocollo TLS/SSL (SSLSocket) per proteggere i dati in transito.
2. **Persistenza Avanzata e Cloud:** Migrazione del database locale verso una soluzione cloud-native o l'introduzione di un framework di ORM (Object-Relational Mapping) come JPA/Hibernate per ottimizzare lo strato di persistenza.
3. **Evoluzione della UI:** Sviluppo di un client web o mobile che riutilizzi l'intero backend logico e di rete grazie alla modularità architetturale dimostrata.
4. **Stanza di gioco:** Transizione da un modello **two-client-one-game-only** a un modello **multiple-clients-multiple-games** in cui possono essere aperte più partite contemporaneamente, tramite meccanismi di concorrenza più avanzati e la gestione asincrona di ciascuna partita, facendo attenzione a non violare i vincoli del modello dati.

In conclusione, il lavoro svolto dimostra come un'attenta progettazione UML (Diagrammi delle Classi e di Sequenza) consenta di governare la complessità di un sistema distribuito, portando alla realizzazione di un software manutenibile, efficiente e pronto per un eventuale contesto di produzione.